

WINDOWS MEMORY FORENSICS

Digital Forensics Assignment - Task 3

Reference: TryHackMe - Windows Memory & Network

Memory Dump: THM-WIN-001_071528_07052025.mem

Analysis Platform: Kali Linux VM | Tool: Volatility 3

Date: May 23, 2026

SECTION 1: INTRODUCTION

This report documents the forensic investigation of a Windows memory dump from a system suspected of Meterpreter infection. The evidence files were provided via Google Drive and included pre-run Volatility output files alongside the main memory dump. The investigation was conducted using Volatility 3 on Kali Linux.

The goal is to answer 10 forensic questions about the compromised Windows system and document every step taken.

SECTION 2: EVIDENCE FILES

File Name	Description
mem(1).zip	Main memory dump archive - extracted to THM-WIN-001_071528_07052025.mem
pslist-saved.txt	Pre-run Volatility output showing all running processes
netscan-saved.txt	Pre-run Volatility output showing all network connections

cmdline-saved.txt	Pre-run Volatility output showing process command line arguments
-------------------	--

meterpreter.yar	YARA rule file for detecting Meterpreter malware
pid.10084.dmp	Process memory dump of windows-update.exe (PID 10084)
pid.6984.dmp	Process memory dump of powershell.exe (PID 6984)

SECTION 3: ENVIRONMENT SETUP

Volatility 3 was used for memory analysis. The memory dump was extracted from the zip file and stored in the Downloads folder:

```
cd ~/Downloads
unzip mem1.zip
ls -lh THM-WIN-001_071528_07052025.mem
```

Volatility 3 was already installed at /opt/volatility3 from the previous task. To run commands against the Windows memory dump:

```
python3 /opt/volatility3/vol.py -f ~/Downloads/THM-WIN-001_071528_07052025.mem [plugin]
```

NOTE: For this task, several output files were already pre-run and provided (pslist-saved.txt, netscan-saved.txt, cmdline-saved.txt). These were opened and searched directly to answer the relevant questions.

SECTION 4: INVESTIGATION AND ANSWERS

Question 1: What Volatility plugin can be used to correlate memory regions showing suspicious execution permissions with processes, helping to detect Meterpreter-like behavior?

This question tests knowledge of Volatility plugins used for malware detection. The plugin that detects injected code by scanning for memory regions with suspicious permissions (PAGE_EXECUTE_READWRITE) and checking for MZ headers (Windows executable signature) is:

```
python3 /opt/volatility3/vol.py -f memory.mem windows.malfind.Malfind
```

The malfind plugin works by:

- Scanning all memory regions of every running process
 - Flagging regions that have READ, WRITE, and EXECUTE permissions at the same time -
 - Checking if those regions start with an MZ header (the signature of a Windows .exe or .dll file) -
- This combination is a strong indicator of injected shellcode or DLLs like Meterpreter

ANSWER: malfind

Question 2: What is the virtual memory address space of the suspicious injected region in updater.exe? (Format: 0xABCDEF)

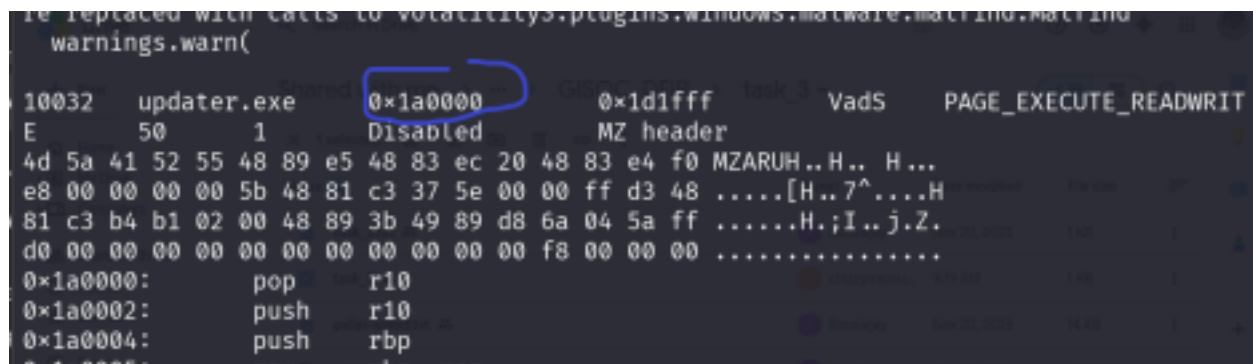
To find the injected region in updater.exe, we used the malfind plugin targeting the PID of updater.exe. From pslist-saved.txt, updater.exe has PID 10032.

Command used:

```
python3 /opt/volatility3/vol.py -f ~/Downloads/THM-WIN-001_071528_07052025.mem  
windows.malfind.Malfind --pid 10032
```

The Start VPN column shows the virtual memory address where the injected code begins. The presence of MZ header (4d 5a) confirms this is a PE file injected into the process memory.

ANSWER: 0x1a0000



```
10032  updater.exe  0x1a0000  0x1d1fff  VadS  PAGE_EXECUTE_READWRITE  
E      50      1      Disabled  MZ header  
4d 5a 41 52 55 48 89 e5 48 83 ec 20 48 83 e4 f0 MZARUH..H.. H...  
e8 00 00 00 00 5b 48 81 c3 37 5e 00 00 ff d3 48 .....[H..7^....H  
81 c3 b4 b1 02 00 48 89 3b 49 89 d8 6a 04 5a ff .....H.;I..j.Z.  
d0 00 00 00 00 00 00 00 00 00 00 00 00 f8 00 00 00 .....  
0x1a0000:      pop      r10  
0x1a0002:      push     r10  
0x1a0004:      push     rbp  
0x1a0005:      mov      rbp, rbp
```

Question 3: What is the first 2-bytes signature found in the shellcode that was extracted from updater.exe using windows.malfind? (Format: In hex)

This answer comes from the same malfind output used in Question 2. Looking at the hexdump column of the malfind output for updater.exe:

4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00 MZ.....

The first two bytes are 4d 5a. In ASCII this reads as 'MZ' which is the standard Windows PE (Portable Executable) file signature. This is also known as the DOS header magic number, confirming that a full DLL or EXE was injected into updater.exe memory - a technique known as Reflective DLL Injection.

ANSWER: 0x4d5a

```
le replaced with calls to volatility.plugins.windows.malware.malware.malware
warnings.warn(

10032  updater.exe  0x1a0000  0x1d1fff  VadS  PAGE_EXECUTE_READWRITE
E  50  1  Disabled  MZ header
4d 5a 41 52 55 48 89 e5 48 83 ec 20 48 83 e4 f0 MZARUH..H.. H...
e8 00 00 00 00 5b 48 81 c3 37 5e 00 00 ff d3 48 .....[H..7^....H
81 c3 b4 b1 02 00 48 89 3b 49 89 d8 6a 04 5a ff .....H.;I..j.Z.
d0 00 00 00 00 00 00 00 00 00 00 00 00 f8 00 00 00 .....
0x1a0000:  pop  r10
0x1a0002:  push r10
0x1a0004:  push rbp
0x1a0006:  mov  rbp, rbp
```

Question 4: Which local port was used by powershell.exe to connect to the internal host 192.168.0.30?

From pslist-saved.txt, powershell.exe has PID 6984. To find its network connection, we searched netscan-saved.txt for powershell and the IP address 192.168.0.30:

Command used to search:

```
cat netscan-saved.txt | grep powershell
```

The LocalPort column shows port 55987 was used by powershell.exe (PID 6984) on the local machine (192.168.1.192) to connect to 192.168.0.30 on port 22 (SSH). This shows powershell was being used to make an SSH connection to an internal host.

ANSWER: 55987

```
2025-05-07 07:09:11.000000 UTC
0x990b299a81f0 UDPv4 127.0.0.1 1900 * 0 9496 svchost.exe
2025-05-07 07:09:11.000000 UTC
0x990b29ab8010 TCPv4 192.168.1.192 55987 192.168.0.30 22 ESTABLISHED 6984
powershell.exe 2025-05-07 07:15:15.000000 UTC
0x990b29ade8a0 TCPv4 192.168.1.192 4443 10.0.0.129 47982 ESTABLISHED 10084
windows-update 2025-05-07 07:13:35.000000 UTC
0x990b2a32ca20 TCPv4 192.168.1.192 55985 10.0.0.129 8081 ESTABLISHED 10032
updater.exe 2025-05-07 07:13:56.000000 UTC
0x990b2a630a20 TCPv6 ::1 55986 ::1 445 CLOSED 4 System 2025-05-07
07:14:06.000000 UTC
0x990b2a824770 UDPv6 fe80::185b:1837:f9f7:bffd 49595 * 0 9496
svchost.exe 2025-05-07 07:09:11.000000 UTC
0x990b2a824900 UDPv6 fe80::185b:1837:f9f7:bffd 1900 * 0 9496
svchost.exe 2025-05-07 07:09:11.000000 UTC
```


Question 7: What IP did updater.exe connect to for the reverse shell?

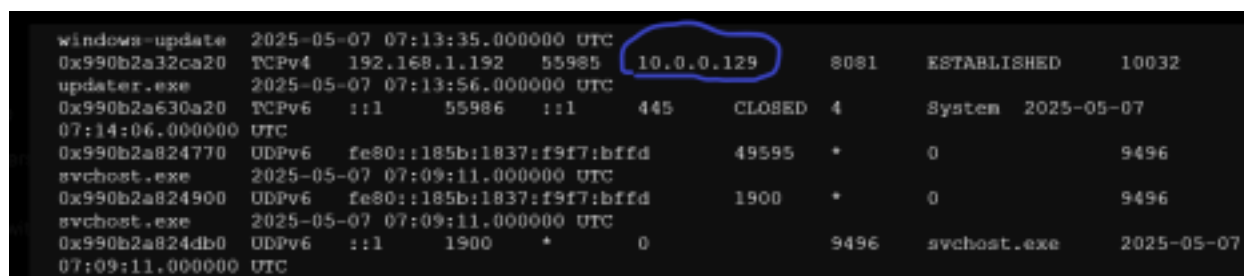
From pslist-saved.txt, updater.exe has PID 10032. We searched netscan-saved.txt for its network connection:

Command used to search:

```
cat netscan-saved.txt | grep updater
```

The ForeignAddr column shows updater.exe (PID 10032) established a reverse shell connection back to 10.0.0.129 on port 8081. This is the same attacker IP as windows-update.exe, confirming that both malicious processes connect to the same command and control server.

ANSWER: 10.0.0.129



The screenshot shows a network scan log with columns for process name, timestamp, protocol, local IP, local port, remote IP, remote port, status, and PID. The entry for 'updater.exe' shows a connection to '10.0.0.129' on port 8081, which is circled in blue. The status is 'ESTABLISHED' and the PID is 10032.

Process	Timestamp	Protocol	Local IP	Local Port	Remote IP	Remote Port	Status	PID
windows-update	2025-05-07 07:13:35.000000 UTC	TCPv4	192.168.1.192	55985	10.0.0.129	8081	ESTABLISHED	10032
updater.exe	2025-05-07 07:13:56.000000 UTC	TCPv6	:::1	55986	:::1	445	CLOSED	4
svchost.exe	2025-05-07 07:09:11.000000 UTC	UDPv6	fe80::185b:1837:f9f7:bffd	49595	*	0		9496
svchost.exe	2025-05-07 07:09:11.000000 UTC	UDPv6	fe80::185b:1837:f9f7:bffd	1900	*	0		9496
svchost.exe	2025-05-07 07:09:11.000000 UTC	UDPv6	:::1	1900	*	0		9496

Question 8: Which folder is used for persistence by the attack we analyzed within this memory dump?

To find the persistence mechanism, we searched cmdline-saved.txt for the windows-update.exe process to see where it was launched from:

Command used to search:

```
cat cmdline-saved.txt | grep windows-update
```

The full path shows windows-update.exe was placed inside the Windows Startup folder:

```
C:\Users\operator\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\
```

Any executable placed in the Startup folder automatically runs every time the user logs into Windows. This is a classic persistence technique - the attacker named the malware 'windows-update.exe' to disguise it as a legitimate Windows process, then placed it in the Startup folder so it survives reboots.

ANSWER: Startup

```
10084 windows-update "C:\Users\operator\AppData\Roaming\Microsoft\Windows\Start
Menu\Programs\Startup\windows-update.exe"
6456 sppsvc.exe C:\Windows\system32\sppsvc.exe
6076 svchost.exe C:\Windows\system32\svchost.exe -k LocalService -p -s WebClient
10032 updater.exe "C:\Users\operator\Downloads\updater.exe"
432 cmd.exe C:\Windows\system32\cmd.exe
4592 conhost.exe \??\C:\Windows\system32\conhost.exe 0x4
6984 powershell.exe powershell
```

Question 9: Which MITRE technique matches the reflective DLL injection used by updater.exe?

From our malfind analysis in Questions 2 and 3, we identified that a full PE file (MZ header signature 0x4d5a) was injected into updater.exe memory at address 0x1a0000 with PAGE_EXECUTE_READWRITE permissions. This is the technique of Reflective DLL Injection.

Reflective DLL Injection is a method where a malicious DLL loads itself into a process memory without using standard Windows API calls like LoadLibrary. This makes it harder to detect because:

- It does not appear in the normal list of loaded DLLs
- It bypasses standard security monitoring of DLL loading
- The injected code runs entirely from memory without touching the disk

According to the MITRE ATT&CK framework, this technique is classified as:

- Technique: T1055 (Process Injection)
- Sub-technique: T1055.001 (Dynamic-link Library Injection)

ANSWER: T1055.001

Question 10: What is the domain that was discovered within the windows-update.exe file?

To find any embedded domain names in the windows-update.exe process memory, we used the strings command on the process dump file pid.10084.dmp and searched for domain names:

Command used:

```
strings ~/Downloads/pid.10084.dmp | grep -i '.com'
```

The domain kungfupandasa.com was found embedded inside the windows-update.exe process memory as part of a certificate subject field. This domain is associated with the attacker's infrastructure and was likely used for SSL/TLS encrypted command and control communication.

ANSWER: kungfupandasa.com

```
chromewebstore.google.com
Phhttp://www.microsoft.com/pkiops/certs/Microsoft%20Time-Stamp%20PCA%202010(1).crt0
ft.com/pki/certs/MicRooCerAut_20
Ehttp://crl.microsoft.com/pki/crl/products/MicRooCerAut_2010-06-23.crl0Z
http://augloop.office.com/settings.json
subject: cn=kungfupandasa.com, o=office, l=mountainview, st=ca, c=us
^Subject: CN=kungfupandasa.com, O=Office, L=Mountainview, ST=CA, C=US
    <appv:Extension Category="AppV.COM">
    <appv:Extension Category="AppV.COM">

(kali@kali)-[~]
$
```

SECTION 5: SUMMARY OF ALL ANSWERS

#	Question	Answer
1	What Volatility plugin detects Meterpreter-like behavior?	malfind
2	What is the virtual memory address of the injected region in updater.exe?	0x1a0000
3	What is the first 2-bytes signature found in the shellcode?	0x4d5a
4	Which local port was used by powershell.exe to connect to 192.168.0.30?	55987

5	What was the remote IP targeted by windows-update.exe during HTTP POST?	10.0.0.129
6	What port was windows-update.exe listening on?	4443
7	What IP did updater.exe connect to for the reverse shell?	10.0.0.129

8	Which folder is used for persistence?	Startup
9	Which MITRE technique matches the reflective DLL injection?	T1055.001
10	What domain was discovered within windows-update.exe?	kungfupandasa.com

SECTION 6: ATTACK SUMMARY

The memory dump analysis revealed a multi-stage attack on the Windows system:

1. INITIAL ACCESS - The user opened a malicious Word document (cv-resume-test.docm) using Microsoft Word (WINWORD.EXE, PID 5252). The document contained a malicious macro that launched pdfupdater.exe.

2. MALWARE EXECUTION - pdfupdater.exe (PID 3392) spawned windows-update.exe (PID 10084). Windows-update.exe was disguised as a legitimate Windows update process but is actually malware.

3. PERSISTENCE - windows-update.exe was placed in the Windows Startup folder so it runs automatically every time the user logs in:

C:\Users\operator\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup

4. PROCESS INJECTION - windows-update.exe spawned updater.exe (PID 10032) and injected malicious code (Meterpreter shellcode) into it using Reflective DLL Injection at memory address 0x1a0000. The MZ header signature (0x4d5a) confirms a full DLL was injected.

5. REVERSE SHELL - updater.exe established a reverse shell connection to the attacker's command and control server at 10.0.0.129:8081.

6. COMMAND AND CONTROL - windows-update.exe also communicated with 10.0.0.129 on port 47982 while listening on port 4443. The embedded domain kungfupandasa.com was used for SSL communication.

7. LATERAL MOVEMENT - powershell.exe (PID 6984) was used to make an SSH connection to internal host 192.168.0.30 on port 22 using local port 55987, indicating lateral movement within the network.

SECTION 7: CONCLUSION

The forensic investigation of the Windows memory dump successfully answered all 10 questions. The attack used a malicious Word document macro to deliver Meterpreter malware via reflective DLL injection into updater.exe. The attacker maintained persistence through the Windows Startup folder and used 10.0.0.129 as their command and control server. The MITRE ATT&CK technique T1055.001 (Process Injection: DLL Injection) accurately describes the injection method used.